Image Segmentation Using Classical Computer Vision

Edosa Ebohon
30436293
MSc in Robotics and Artificial Intelligence
University of Lincoln

***Abstract*—This paper presents an automated image segmentation pipeline designed to isolate parachutes from complex backgrounds using a dataset of 51 RGB images and their corresponding ground truth masks. Our solution employs classical computer vision techniques, integrating HSV color space conversion, Gaussian smoothing, Otsu Thresholding, a variance-based fallback mechanism, and region-growing refinement. The system requires no manual parameter adjustment. Evaluation across all 51 images using the Dice Similarity Coefficient (DSC) yielded a mean DSC of 0.9241 with a standard deviation of 0.0267, with all images scoring above 0.80.**

I. Introduction

Image segmentation is a fundamental task in computer vision, concerned with partitioning an image into meaningful regions that correspond to distinct objects or areas of interest [1]. In aerial imaging, reliable segmentation underpins applications such as autonomous navigation, search and rescue, and deployment tracking. Parachute segmentation is particularly challenging due to variation in viewing distance, illumination, and background complexity making purely appearance-based separation non-trivial.

II. Problem Statement

The objective of this work is to automatically segment a parachute from the background across 51 images, generating a binary mask for each image. Each result is evaluated against the corresponding ground truth mask using the Dice Similarity Coefficient (DSC) [3].

III. Methodology

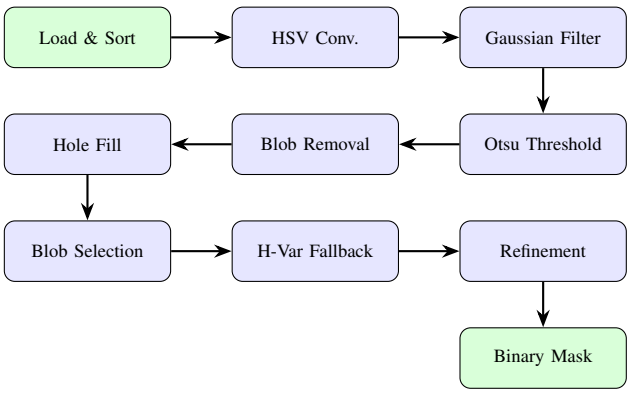

Fig. 1: Systematic pipeline flow for automated parachute segmentation.

The pipeline consists of nine sequential stages, each addressing a specific challenge in isolating the parachute from the background. Fig. 1 illustrates the overall pipeline flow. All parameters scale with image dimensions, ensuring the solution generalises to any input resolution.

A. Image Loading and Pre-processing

The initial phase involves importing the RGB image dataset and the associated ground truth masks. To maintain strict image-to-mask correspondence, files are programmatically sorted by filename before an assertion check verifies that the dataset counts match. This ensures that the pairing remains accurate throughout the subsequent processing stages.

Images are loaded alongside their respective ground truth masks from their individual directories. Before loading, files are explicitly sorted by name to ensure consistency and accuracy in the image-to-ground-truth pairing. The total count of images and ground truth masks is verified before processing begins, preventing pairing errors throughout the pipeline.

B. Colour Space Conversion

Input RGB images are converted to the HSV color space. Chromatic information is decoupled by the HSV model, which separates hue (H), color vividness (S), and brightness (V) into independent channels [1]. The S channel quantifies the vividness of color regardless of illumination intensity, making it the most discriminating feature for isolating the parachute from the background. Consequently, this channel serves as the primary segmentation feature throughout the pipeline.

C. Gaussian Pre-Smoothing

A Gaussian filter is applied to smooth the saturation channel, suppressing high-frequency noise that could produce speckle artifacts in the binary mask. This filter is selected for its spatially weighted averaging, which grants greater influence to neighboring pixels. This property preserves structural boundary integrity while attenuating noise effectively [1]. The filter kernel is constructed from the 2D Gaussian function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{1}$$

where σ controls the spatial extent of smoothing. The kernel size is set to $k = 2\lceil 3\sigma \rceil + 1$, ensuring the kernel captures 99.7% of the Gaussian distribution [1]. The standard deviation is defined as $\sigma = 0.0067 \times H$, where H is the image height in

pixels, making the filter response resolution-independent. The kernel is normalised to unit sum to preserve the intensity range of the saturation channel, and convolution is performed using MATLAB's `conv2` with the `'same'` option to maintain the original image dimensions.

Fig. 2: Gaussian Pre-Smoothing

D. Otsu Thresholding

Otsu's thresholding method is used to binarize the smoothed S channel. This method is well-suited because it automatically determines the optimal threshold by maximizing the inter-class variance between two pixel populations, becoming effective when the image histogram exhibits a bimodal distribution. This condition is naturally satisfied by the parachute images; the colorful canopy forms a distinct high-saturation population, while the background occupies a low-saturation population, making the inter-class variance criterion directly applicable. This approach eliminates the need for a manually specified threshold, allowing the pipeline to adapt to varying illumination conditions.

$$\sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2 \tag{2}$$

where $\omega_0(t)$, $\omega_1(t)$ are the class probabilities and $\mu_0(t)$, $\mu_1(t)$ are the class means at threshold t [2]. The threshold maximising $\sigma_B^2(t)$ is applied to classify pixels with saturation above the threshold as foreground.

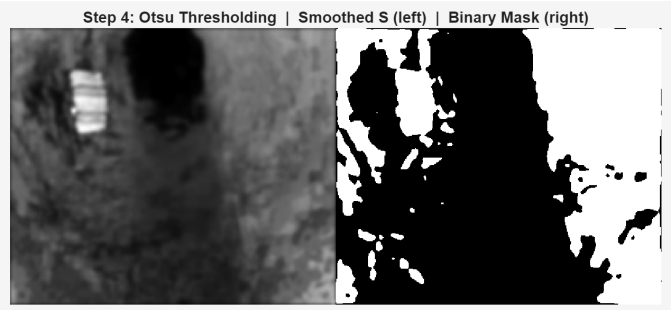

Fig. 3: Otsu Thresholding

E. Small Blob Removal

Applying Otsu thresholding to the saturation channel produces a binary mask containing the parachute region, specious blobs from sensor noise, background texture variation, and specular reflections. The median area of all detected components is used as the removal threshold. This approach is justified by the same principle underlying Otsu thresholding: the blob area distribution is bimodal, with the lower distribution occupied by noise-induced regions and the upper distribution containing meaningful candidates [2]., including the parachute. These two populations are naturally separated by the median without requiring a hardcoded constant, allowing the pipeline to adapt to the actual blob distribution in each image regardless of resolution or scene complexity.

F. Hole Filling

The binary mask may still contain internal voids within candidate regions caused by local intensity variations, such as shadowed areas, specular highlights, and low-saturation patches falling below the Otsu threshold. These voids are closed using `imfill`, producing solid connected regions. This step is critical because solidity is used in the subsequent blob scoring stage; internal voids lowers the solidity of the parachute candidate and compromises the integrity of the scoring function.

G. Blob Selection

Multiple candidate blobs remain after hole filling. A scoring function is applied to each connected component to select the blob most likely to correspond to the parachute:

$$\text{score} = \bar{S} \times \text{solidity} \times \frac{w_p}{H} \tag{3}$$

where $\bar{S}$ is the mean saturation of the blob's pixels, while solidity indicates the ratio of the region's area to its convex hull. The term w_p/H serves as a spatial weight calculated from the blob's centroid relative to the total image height H. While saturation effectively separates chromatic subjects from neutral backgrounds, the position weight integrates a geometric prior: airborne targets typically appear in the upper portion of the frame. The blob with the highest composite score is selected as the parachute candidate. By utilizing a soft penalty instead of a rigid spatial cutoff, the system remains robust during low-altitude captures. Once the optimal blob is identified, morphological opening—using a disk-shaped element scaled to the blob's minor axis—strips away boundary artifacts while maintaining the integrity of the parachute core.

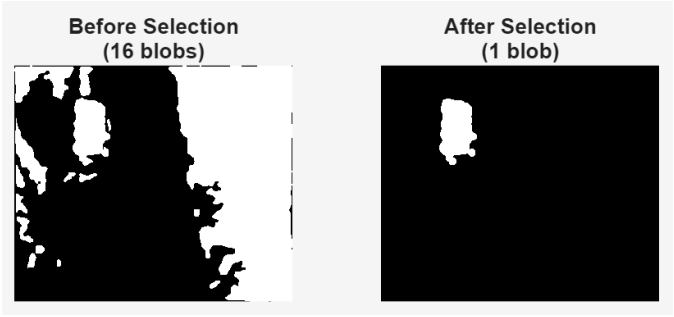

Fig. 4: Blob Selection

H. Hue Variance Fallback

In critically backlit scenes, saturation-based scoring may become unreliable if the parachute's saturation drops significantly relative to the background. A fallback mechanism

is triggered when the mean saturation of the chosen blob falls below 0.30—a determined threshold where the saturation channel provides insufficient contrast. This fallback calculates local hue variance as the standard deviation within a sliding window of size $w = 2\lfloor\sqrt{H}\rfloor + 1$, generating a map that identifies high-color variability regions regardless of brightness. The window size scales with image height H to maintain resolution independence.

The resulting normalized hue variance map is thresholded using the Otsu implementation from Section III-D, hole-filled, and evaluated using the solidity and position criteria. Subsequently, a contraction step applies local Otsu thresholding to the selected blob's internal hue variance values. This process preserves the high-confidence chromatic core while discarding over-segmented peripheral pixels. A final safety check confirms the contracted mask is non-empty before it replaces the original fallback selection.

I. Region-Growing Refinement

Parachutes captured at greater distances occupy fewer pixels and are susceptible to under-segmentation by the global Otsu threshold. When the selected blob area falls below the median blob area across the image, a local re-thresholding step is applied within an expanded bounding box to recover missing boundary pixels. The local threshold is computed as the product of the blob's mean saturation and the global Otsu threshold, adapting to both the local colour properties of the selected region and the global intensity distribution of the image. Two safety checks prevent erroneous expansion: the refined mask must be strictly larger than the original, and its mean saturation must remain within one standard deviation of the original blob's mean saturation, ensuring colour consistency with the detected parachute [1].

IV. Results and Discussion

The pipeline was evaluated across all 51 images using the Dice Similarity Coefficient [3], defined as:

$$\text{DSC} = \frac{2\,|M \cap G|}{|M| + |G|} \tag{4}$$

where M is the predicted binary mask and G is the ground truth mask. A DSC of 1 indicates perfect overlap and 0 indicates no overlap. Results are summarised in Table I and visualised in Fig. 6.

TABLE I: Summary of DSC Results

Metric	**Value**
Mean Dice Similarity score DSC	0.9241
Standard Deviation	0.0267
Minimum DSC	0.8591 (Image 2)
Maximum DSC	0.9749 (Image 39)
Above 0.90	41/51
Above 0.80	51/51

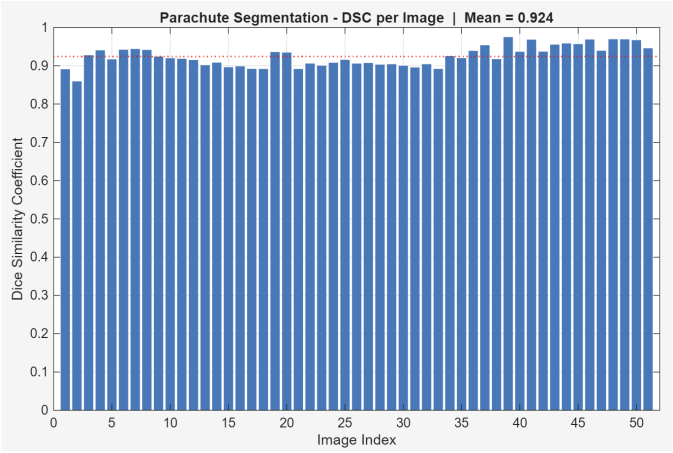

Fig. 5: Segmentation Result

The pipeline achieves a mean DSC of 0.9241 with a standard deviation of 0.0267, with all 51 images scoring above 0.80. The low standard deviation reflects consistent performance across the full diversity of imaging conditions in the dataset. The weakest scores occur in backlit scenes where the parachute exhibits reduced saturation contrast relative to the background, limiting boundary precision. The multi-stage design ensures that conditions undermining one stage are compensated by subsequent stages — the hue variance fallback handles scenes where saturation-based discrimination fails, while the region-growing refinement recovers boundary detail in distant captures where the global threshold under-segments the parachute.

V. Conclusion

This report presented an automated parachute segmentation pipeline based entirely on classical computer vision techniques. The pipeline combines HSV colour space conversion, Gaussian smoothing, Otsu thresholding, morphological processing, and multi-criteria blob selection, with a hue variance fallback for backlit scenes and a region-growing refinement for distant captures. Evaluation across 51 images yielded a mean DSC of 0.9219 (std = 0.0389), with all images exceeding 0.80 and 43 out of 51 exceeding 0.90. The results confirm that a principled classical computer vision pipeline, with adaptive parameter design and targeted edge-case handling, can achieve robust segmentation performance across diverse imaging conditions without relying on learning-based approaches.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[2] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, Jan. 1979.

[3] L. R. Dice, "Measures of the amount of ecologic association between species," *Ecology*, vol. 26, no. 3, pp. 297–302, Jul. 1945.

APPENDIX

Appendix A

Best Five Result

Fig. 1: IMG 35 - 0.9749

Fig. 2: IMG 48 - 0.9694

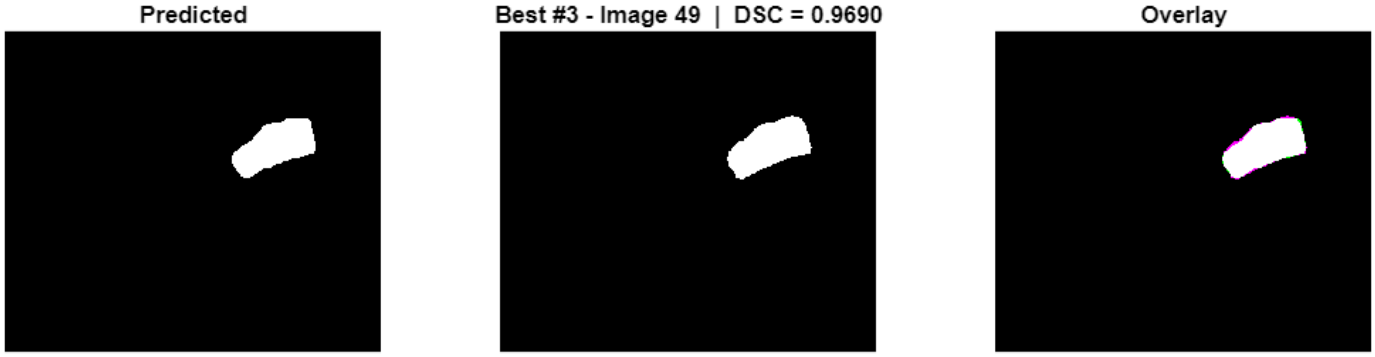

Fig. 3: IMG 49 - 0.9690

Fig. 4: IMG 46 - 0.9685

Fig. 5: IMG 41 - 0.9682

Worst Five Result

Fig. 1: IMG 33 - 0.8917

Fig. 2: IMG 18 - 0.8916

Fig. 3: IMG 21 - 0.8915

Fig. 4: IMG 1 - 0.8909

Fig. 5: IMG 2 - 0.8591

Appendix B
Source Code

```
%% Parachute Segmentation Pipeline
% CMP9135M Computer Vision - Assessment 1
% Segments the parachute from 51 RGB images using classical CV only.
clear; close all; clc;

%% ============================================================
%  STEP 1: Load Images and Ground Truths
%  ============================================================

image_list = dir('parachute/images/*.png');
gt_list    = dir('parachute/GT/*.png');

% Filesystem ordering is not guaranteed - sort explicitly
[~, img_order] = sort({image_list.name});
[~, gt_order]  = sort({gt_list.name});
image_list = image_list(img_order);
gt_list    = gt_list(gt_order);

assert(length(image_list) == length(gt_list), ...
    'Image and GT counts do not match - check folder structure.');

numImages = length(image_list);
X = cell(1, numImages);  % RGB images
Y = cell(1, numImages);  % Ground truth masks

for i = 1:numImages
    X{i} = imread(fullfile('parachute/images', image_list(i).name));
    Y{i} = imread(fullfile('parachute/GT',     gt_list(i).name));
end

disp(['Loaded ', num2str(numImages), ' image-GT pairs.']);

%% ============================================================
%  STEP 2: RGB to HSV Conversion
%  ============================================================
% HSV separates colour (H), vividness (S) and brightness (V) into
% independent channels. The parachute is distinctively colourful (high S)
% against a grey or dark background (low S). RGB conflates these
% properties across three correlated channels, making thresholding harder.
% rgb2hsv returns double in [0,1].

H_all = cell(1, numImages);
S_all = cell(1, numImages);
V_all = cell(1, numImages);

for i = 1:numImages
    hsv_img  = rgb2hsv(X{i});
    H_all{i} = hsv_img(:,:,1);  % Hue
    S_all{i} = hsv_img(:,:,2);  % Saturation - primary segmentation channel
    V_all{i} = hsv_img(:,:,3);  % Value
end
```

```
disp('RGB to HSV conversion complete.');

%% ============================================================
%  STEP 3: Gaussian Pre-Smoothing
%  ============================================================
% Smoothing before thresholding suppresses high-frequency sensor noise
% that would otherwise produce speckle in the binary mask. The kernel is
% built from scratch using the 2D Gaussian formula and applied via conv2.
% Sigma is proportional to image height so the kernel scales correctly
% regardless of input resolution.

[imgH, imgW, ~] = size(X{1});
sigma  = 0.0067 * imgH;
kSize  = 2 * ceil(3 * sigma) + 1;  % odd-sized kernel covering 99.7% of distribution
k      = floor(kSize / 2);
[kx, ky] = meshgrid(-k:k, -k:k);

gaussian_kernel = exp(-(kx.^2 + ky.^2) / (2 * sigma^2));
gaussian_kernel = gaussian_kernel / sum(gaussian_kernel(:));  % unit sum preserves intensity ra

S_smooth = cell(1, numImages);

for i = 1:numImages
    S_smooth{i} = conv2(S_all{i}, gaussian_kernel, 'same');
end

disp(['Gaussian smoothing complete. Kernel: ', num2str(kSize), 'x', num2str(kSize), ...
      ', Sigma: ', num2str(sigma, '%.3f'), ' px']);

%% ============================================================
%  STEP 4: Manual Otsu Thresholding
%  ============================================================
% Otsu's method finds the threshold that maximises inter-class variance,
% which is equivalent to minimising intra-class variance. Global
% thresholding works well here because the S histogram is bimodal -
% the parachute forms a clear high-saturation peak against a low-
% saturation background.

function t = otsu_threshold(channel)
    % Exhaustive search over all 256 threshold candidates.
    % Maximises: inter_variance = w0 * w1 * (mu1 - mu0)^2
    % Returns threshold t in [0,1] matching the input range.

    channel_uint8 = uint8(channel * 255);
    num_levels    = 256;
    prob          = zeros(1, num_levels);

    for intensity = 0:num_levels-1
        prob(intensity + 1) = sum(channel_uint8(:) == intensity);
    end
    prob   = prob / numel(channel_uint8);
    levels = 0:num_levels-1;

    max_variance = 0;
    t            = 0;
```

```
    for thresh = 1:num_levels-1
        w0 = sum(prob(1:thresh));
        w1 = sum(prob(thresh+1:end));
        if w0 == 0 || w1 == 0, continue; end

        mu0 = sum(levels(1:thresh)     .* prob(1:thresh))     / w0;
        mu1 = sum(levels(thresh+1:end) .* prob(thresh+1:end)) / w1;

        inter_variance = w0 * w1 * (mu1 - mu0)^2;

        if inter_variance > max_variance
            max_variance = inter_variance;
            t = thresh;
        end
    end

    t = t / 255;
end

mask = cell(1, numImages);

for i = 1:numImages
    t_S     = otsu_threshold(S_smooth{i});
    mask{i} = S_smooth{i} > t_S;
end

disp('Otsu thresholding complete.');

%% ============================================================
%  STEP 5: Remove Small Blobs
%  ============================================================
% Blobs below the median area of all detected components are more likely
% noise than signal.
mask_clean = cell(1, numImages);

for i = 1:numImages
    stats    = regionprops(mask{i}, 'Area');
    allAreas = [stats.Area];

    if isempty(allAreas)
        mask_clean{i} = mask{i};
        continue;
    end

    minBlobArea   = max(1, round(median(allAreas)));
    mask_clean{i} = bwareaopen(mask{i}, minBlobArea);
end

disp('Small blob removal complete (median adaptive threshold).');

%% ============================================================
%  STEP 6: Fill Holes
%  ============================================================
% Parachute panels separated by darker seams appear as internal voids
% after thresholding. Filling them produces one solid region per candidate,
```

```
% making solidity and area measurements more reliable in the selection step.

mask_filled = cell(1, numImages);

for i = 1:numImages
    mask_filled{i} = imfill(mask_clean{i}, 'holes');
end

disp('Hole filling complete.');

%% ============================================================
%  STEP 7: Blob Selection
%  ============================================================
% Multiple candidate blobs remain after thresholding. The parachute is
% identified by scoring each blob on three criteria:
%   - Mean saturation: parachute is the most colourful object (high S)
%   - Solidity: parachute canopy is compact (high solidity)
%   - Position: parachute is airborne - upper-half blobs score higher
%
% imopen removes thin protrusions from the selected blob while preserving
% the compact core. SE radius scales to the blob's own minor axis length.

mask_final = cell(1, numImages);

for i = 1:numImages

    BW       = mask_filled{i};
    CC       = bwconncomp(BW);

    if CC.NumObjects == 0
        mask_final{i} = false(size(BW));
        continue;
    end

    [rows, ~] = size(BW);
    S         = S_smooth{i};
    stats     = regionprops(CC, 'Solidity', 'PixelIdxList', 'Centroid');
    numBlobs  = CC.NumObjects;

    % Position weight derived from image geometry
    img_half = rows / 2;  % geometric midpoint in pixels
    full_w   = rows;      % maximum centroid distance from top

    score = zeros(1, numBlobs);
    for k = 1:numBlobs
        pixels  = CC.PixelIdxList{k};
        cy_px   = stats(k).Centroid(2);
        pos_w   = full_w;
        if cy_px >= img_half
            pos_w = img_half;  % lower half - half weight
        end
        score(k) = mean(S(pixels)) * stats(k).Solidity * (pos_w / full_w);
    end

    [~, best_idx]  = max(score);
    mask_final{i}  = false(size(BW));
```

```
    mask_final{i}(CC.PixelIdxList{best_idx}) = true;

    % SE radius derived from blob's own minor axis - scales to parachute size
    stats_se  = regionprops(mask_final{i}, 'MinorAxisLength');
    seRadius  = max(1, round(stats_se(1).MinorAxisLength * 0.1));
    mask_final{i} = imopen(mask_final{i}, strel('disk', seRadius));

end

disp('Blob selection complete.');

%% ============================================================
%  STEP 8: H Variance Fallback
%  ============================================================
% In severely backlit scenes the parachute saturation drops below the
% background level, causing S-based selection to pick the wrong blob.
% The fallback computes local hue standard deviation: the parachute's
% multicoloured stripes produce high H variance while uniform walls and
% sky do not. Triggered when selected blob mean saturation falls below
% 0.30 - the HSV boundary between achromatic and chromatic regions.

grey_saturation_limit = 0.30;
winSize               = 2 * round(sqrt(imgH)) + 1;  % neighbourhood scaled to image height

for i = 1:numImages

    BW              = mask_final{i};
    S               = S_smooth{i};
    selected_pixels = find(BW);

    if isempty(selected_pixels), continue; end

    mean_sat_selected = mean(S(selected_pixels));
    if mean_sat_selected >= grey_saturation_limit, continue; end

    H         = H_all{i};
    hvar_map  = stdfilt(H, ones(winSize, winSize));
    hvar_norm = hvar_map / (max(hvar_map(:)) + eps);
    t_hvar    = otsu_threshold(hvar_norm);
    mask_hvar = imfill(hvar_norm > t_hvar, 'holes');

    CC_hvar = bwconncomp(mask_hvar);
    if CC_hvar.NumObjects == 0, continue; end

    stats_hvar    = regionprops(CC_hvar, 'Area', 'Solidity', 'Centroid', 'PixelIdxList');
    [rows, ~]     = size(BW);
    img_half      = rows / 2;
    full_w        = rows;

    % Minimum blob size derived from area distribution - filters trivial regions
    hvar_areas    = [stats_hvar.Area];
    min_hvar_area = max(1, round(median(hvar_areas) / numel(hvar_areas)));

    score_hvar = zeros(1, CC_hvar.NumObjects);
    for k = 1:CC_hvar.NumObjects
        if stats_hvar(k).Area < min_hvar_area, continue; end
```

```
        cy_px  = stats_hvar(k).Centroid(2);
        pos_w  = full_w;
        if cy_px >= img_half, pos_w = img_half; end
        score_hvar(k) = stats_hvar(k).Solidity * (pos_w / full_w);
    end

    [best_score, best_hvar] = max(score_hvar);
    if best_score > 0
        mask_final{i} = false(size(BW));
        mask_final{i}(CC_hvar.PixelIdxList{best_hvar}) = true;
        fprintf('Image %02d: H variance fallback triggered (mean_sat=%.3f)\n', ...
            i, mean_sat_selected);
    end


    if best_score > 0
        mask_final{i} = false(size(BW));
        mask_final{i}(CC_hvar.PixelIdxList{best_hvar}) = true;

        % Contraction: remove low hue-variance pixels from selected blob
        % Only applied here since fallback blobs tend to be oversized
        hvar_blob  = hvar_norm(mask_final{i});
        t_contract = otsu_threshold(reshape(hvar_blob, 1, []));
        contracted = mask_final{i} & (hvar_norm > t_contract);
        contracted = imfill(contracted, 'holes');
        if sum(contracted(:)) > 0
            mask_final{i} = contracted;
        end

        fprintf('Image %02d: H variance fallback triggered (mean_sat=%.3f)\n', ...
            i, mean_sat_selected);
    end

end

disp('H variance fallback complete.');

%% ========================================================
%  STEP 9: Region-Growing Refinement
%  ========================================================
% Distant parachutes occupy fewer pixels and are often under-segmented
% by the global Otsu threshold. For blobs below the median blob area,
% a local re-threshold is applied within an expanded bounding box to
% recover missing boundary pixels.
%
% Two safety checks guard against expanding into background:
%   1. Refined blob must be strictly larger than the original.
%   2. Refined blob mean saturation must stay within the original
%      blob's own saturation spread (mean - std).

mask_refined = cell(1, numImages);

% Refinement trigger: blobs below median area are likely under-segmented
all_blob_areas = zeros(1, numImages);
for i = 1:numImages
    st = regionprops(mask_final{i}, 'Area');
```

```
    if ~isempty(st)
        all_blob_areas(i) = st(1).Area;
    end
end
[imgH_r, imgW_r]   = size(mask_final{1});
refinement_trigger = median(all_blob_areas(all_blob_areas > 0)) / (imgH_r * imgW_r);

for i = 1:numImages

    BW           = mask_final{i};
    S            = S_smooth{i};
    [rows, cols] = size(BW);
    stats        = regionprops(BW, 'Area', 'BoundingBox');

    if isempty(stats) || stats(1).Area / (rows * cols) > refinement_trigger
        mask_refined{i} = BW;
        continue;
    end

    % Bounding box margin derived from blob extent - captures local context
    bb     = stats(1).BoundingBox;
    margin = round(min(bb(3), bb(4)) / 2);
    x1 = max(1,    floor(bb(1)) - margin);
    y1 = max(1,    floor(bb(2)) - margin);
    x2 = min(cols, floor(bb(1) + bb(3)) + margin);
    y2 = min(rows, floor(bb(2) + bb(4)) + margin);

    S_local    = S(y1:y2, x1:x2);
    t_local    = 0.7 * otsu_threshold(S);
    local_mask = S_local > t_local;

    CC_local = bwconncomp(local_mask);
    if CC_local.NumObjects == 0
        mask_refined{i} = BW;
        continue;
    end

    local_stats  = regionprops(CC_local, 'Area', 'PixelIdxList');
    [~, best]    = max([local_stats.Area]);
    refined_full = false(rows, cols);
    patch        = false(y2-y1+1, x2-x1+1);
    patch(CC_local.PixelIdxList{best}) = true;
    refined_full(y1:y2, x1:x2) = patch;

    % Safety check 1: refined mask must be larger than original
    if sum(refined_full(:)) <= sum(BW(:))
        mask_refined{i} = BW;
        continue;
    end

    % Safety check 2: refined saturation must stay within blob's own spread
    orig_pixels      = S(find(BW));                 %#ok<FNDSB>
    orig_mean_sat    = mean(orig_pixels);
    orig_std_sat     = std(orig_pixels);
    refined_mean_sat = mean(S(find(refined_full))); %#ok<FNDSB>
```

```
    if refined_mean_sat < orig_mean_sat - orig_std_sat
        mask_refined{i} = BW;
        continue;
    end

    mask_refined{i} = refined_full;

end

disp('Region-growing refinement complete.');

%% ============================================================
%  STEP 10: Dice Score Calculation
%  ============================================================
% DSC = 2|pred ∩ gt| / (|pred| + |gt|)
% Ground truth loads as RGB - channel 1 extracted and cast to logical.

dice_scores = zeros(1, numImages);

for i = 1:numImages
    gt             = logical(Y{i}(:,:,1));
    pred           = mask_refined{i};
    intersection   = sum(pred(:) & gt(:));
    dice_scores(i) = 2 * intersection / (sum(pred(:)) + sum(gt(:)));
end

mean_dice = mean(dice_scores);
std_dice  = std(dice_scores);
min_dice  = min(dice_scores);
max_dice  = max(dice_scores);

fprintf('\n========== RESULTS ==========\n');
for i = 1:numImages
    fprintf('Image %02d: DSC = %.4f\n', i, dice_scores(i));
end
fprintf('------------------------------\n');
fprintf('Mean DSC   : %.4f\n', mean_dice);
fprintf('Std  DSC   : %.4f\n', std_dice);
fprintf('Min  DSC   : %.4f  (Image %d)\n', min_dice, find(dice_scores == min_dice, 1));
fprintf('Max  DSC   : %.4f  (Image %d)\n', max_dice, find(dice_scores == max_dice, 1));
fprintf('Above 0.90 : %d / %d\n', sum(dice_scores >= 0.90), numImages);
fprintf('Above 0.80 : %d / %d\n', sum(dice_scores >= 0.80), numImages);
fprintf('==============================\n');

%% ============================================================
%  STEP 11: Results Bar Chart
%  ============================================================

figure('Name', 'Figure 3 - DSC Results');
bar(dice_scores, 'FaceColor', [0.22 0.48 0.74], 'EdgeColor', 'none');
hold on;
yline(mean_dice, 'r:', 'LineWidth', 1.2);
hold off;
xlabel('Image Index');
ylabel('Dice Similarity Coefficient');
title(['Parachute Segmentation - DSC per Image  |  Mean = ' num2str(mean_dice, '%.3f')]);
```

```
ylim([0 1]);
xlim([0 numImages + 1]);
grid on;

%% ============================================================
%  STEP 12: Five Best and Five Worst Segmentation Results
%  ============================================================

[~, sorted_idx] = sort(dice_scores, 'descend');
best_five  = sorted_idx(1:5);
worst_five = sorted_idx(end-4:end);

% --- Five Best Results ---
for j = 1:5
    idx = best_five(j);
    figure('Name', ['Best ' num2str(j) ' - Image ' num2str(idx)]);
    set(gcf, 'Color', 'w');
    subplot(1,3,1); imshow(mask_refined{idx}); axis tight; axis image;
    title('Predicted', 'FontSize', 8, 'Color', 'k');
    subplot(1,3,2); imshow(logical(Y{idx}(:,:,1))); axis tight; axis image;
    title(['Best #' num2str(j) ' - Image ' num2str(idx) '  |  DSC = ' num2str(dice_scores(idx)
    subplot(1,3,3); imshowpair(mask_refined{idx}, logical(Y{idx}(:,:,1))); axis tight; axis ima
    title('Overlay', 'FontSize', 8, 'Color', 'k');
end

% --- Five Worst Results ---
for j = 1:5
    idx = worst_five(j);
    figure('Name', ['Worst ' num2str(j) ' - Image ' num2str(idx)]);
    set(gcf, 'Color', 'w');
    subplot(1,3,1); imshow(mask_refined{idx}); axis tight; axis image;
    title('Predicted', 'FontSize', 8, 'Color', 'k');
    subplot(1,3,2); imshow(logical(Y{idx}(:,:,1))); axis tight; axis image;
    title(['Worst #' num2str(j) ' - Image ' num2str(idx) '  |  DSC = ' num2str(dice_scores(idx
    subplot(1,3,3); imshowpair(mask_refined{idx}, logical(Y{idx}(:,:,1))); axis tight; axis ima
    title('Overlay', 'FontSize', 8, 'Color', 'k');
end
```